

Многопоточные вычисления на основе технологий MPI и OpenMP: Взаимодействия типа точка-точка

Н. И. Хохлов

МФТИ, Долгопрудный

15 сентября 2026 г.

Взаимодействия точка-точка. Особенности

- Всего участвуют *два* и только два процесса.
- Явный процесс взаимодействия. Один процесс всегда принимает данные, другой отправляет.
- Существует несколько типов взаимодействия, операции приема/отправки разных типов могут комбинироваться.
- Процессы обмениваются данными, только если состоят в одном коммутаторе (в рамках одного контекста).

Функции отсылки и приема построены по единому интерфейсу.

Example

```
send(address, count, datatype, destination, tag, comm)
```

и

```
recv(address, maxcount, datatype, source, tag, comm, status)
```

Сообщение идентифицируется *конвертом*: (source, tag, communicator).

Порядок и matching сообщений

- Сообщения от **одного** процесса к другому **не обгоняют** друг друга (FIFO при совпадении source/tag/comm).
- Сообщения от **разных** процессов могут приходить в произвольном порядке.
- Прием с `MPI_ANY_SOURCE` / `MPI_ANY_TAG` выбирает любое подходящее сообщение; порядок между разными источниками не гарантирован.
- Тег (tag) позволяет различать логические потоки сообщений между одной парой процессов.

- Четыре типа операций отсылки и одна операция приема.
- Завершение операции гарантирует безопасность дальнейшего использования буфера отсылки (изменения никак не скажутся на стороне получателя).
- Все операции существуют в блокирующем и **неблокирующем** виде.
- Несколько процедур взаимодействия могут быть объединены для ускорения отсылки (persistent communication).

Типы пересылок

Тип взаимодействия	Условие завершения
Синхронная отсылка (synchronous)	Завершается только после начала приема
Буферизованная отсылка (buffered)	Завершается после копирования в пользовательский буфер; прием не гарантирует
Обычная отсылка (standard)	Как синхронная или буферизованная (решает реализация)
Отсылка по готовности (ready)	Корректна, только если Recv уже выставлен; иначе поведение не определено
Прием (Receive)	Завершается, когда сообщение доставлено в буфер приема

Тип взаимодействия	Блокирующая	Неблокирующая
Синхронная отсылка	MPI_Ssend	MPI_Issend
Буферизованная отсылка	MPI_Bsend	MPI_Ibsend
Обычная отсылка	MPI_Send	MPI_Isend
Отсылка по готовности	MPI_Rsend	MPI_Irsend
Прием	MPI_Recv	MPI_Irecv

Стандартная отсылка

- Завершается, как только буфер становится безопасен для дальнейшего использования.
- Не гарантирует доставку и даже того, что начался прием (сообщение может находиться в системном буфере).
- Может быть реализована через синхронную или буферизованную отсылку (или их комбинацию).
- При написании приложений следует рассматривать работу функции **как синхронную**.
- Множество операций отсылок без приемов может загружать сеть и исчерпать буферы.

MPI_Send: eager и rendezvous

Реализации обычно выбирают протокол по размеру сообщения (порог зависит от Open MPI / MPICH / Intel MPI и настроек):

- **Eager** (малые сообщения): данные копируются в системный буфер, MPI_Send может завершиться до вызова Recv.
- **Rendezvous** (крупные сообщения): отправитель ждет готовности приема (handshake), поведение ближе к MPI_Ssend.

Один и тот же код на 8 Б и на 8 МБ может вести себя по-разному — в том числе по риску взаимной блокировки.

```
int MPI_Send(void *buf, int count, MPI_Datatype type, int dest, int tag,  
MPI_Comm comm);
```

- **buf** – адрес начала расположения пересылаемых данных;
- **count** – число пересылаемых элементов;
- **type** – MPI-тип посылаемых элементов;
- **dest** – номер процесса-получателя в группе, связанной с коммуникатором **comm**;
- **tag** – идентификатор сообщения;
- **comm** – коммуникатор.

Типы данных

Типы данных можно создавать для различных типов языка.
Существуют глобальные типы для встроенных типов языка.

Тип C	Тип MPI
int	MPI_INT
float	MPI_FLOAT
char	MPI_CHAR
double	MPI_DOUBLE
long	MPI_LONG
long double	MPI_LONG_DOUBLE
*	MPI_BYTE
*	MPI_PACKED

Передача числа

```
int a;  
...  
MPI_Send(&a, 1, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

Передача статического массива

```
int a[5];  
...  
MPI_Send(a, 5, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

или

```
MPI_Send(&a[0], 5, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

Передача динамического массива

```
int *a = (int*)malloc(sizeof(int) * 5);  
...  
MPI_Send(a, 5, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

Передача части статического или динамического массива

```
int a[5]; (или int *a = (int*)malloc(sizeof(int) * 5);)  
...  
MPI_Send(a+1, 2, MPI_INT, rank, tag, MPI_COMM_WORLD);
```

Будут отправлены элементы с индексами 1 и 2 (два элемента).

Синхронная отсылка

- Завершение гарантирует начало приема.
- Не гарантирует, что операция приема завершена.
- Синтаксис аналогичен MPI_Send.
- Несет меньшую нагрузку на сеть, чем MPI_Send (нет лишнего буферирования), но в ряде случаев может работать медленнее.
- Необходимо следить за возможными блокировками.

Буферизованная отсылка

- Завершение гарантирует безопасность буфера отсылаемого сообщения.
- Не гарантирует прием.
- Синтаксис аналогичен MPI_Send.
- Пользователь должен явно управлять буфером для сообщений.
- В прикладном коде используется редко: проще и надежнее неблокирующие операции или MPI_Sendrecv.

Подключение буфера

```
int MPI_Buffer_attach (void* buffer, int size);
```

- **buffer** – адрес начала буфера, должен быть выделен пользователем заранее;
- **size** – размер буфера в байтах.

Отключение буфера

```
int MPI_Buffer_detach(void* buffer_addr, int* size);
```

- **buffer_addr** – указатель на буфер, который используется. Фактически это void**;
- **size** – размер буфера в байтах.

Если пользователь не подключил буфер, то MPI считает, что его размер нулевой.

Что будет, если пользователь вызвал `MPI_Buffer_detach`, а сообщения еще не доставлены?

Функция будет завершена, когда завершатся все операции буферизованных пересылок.

Работа с буфером сообщений

Как узнать размер буфера, который надо выделить?

Необходимо выделить столько места, чтобы туда поместились все отсылаемые сообщения + некое дополнительное пространство. Верхний предел дополнительного пространства описывается константой **`MPI_BSEND_OVERHEAD`**.

Для определения размера сообщения, занимаемого в буфере, существует функция

```
int MPI_Pack_size(int incount, MPI_Datatype datatype, MPI_Comm comm, int *size);
```

- **incount** – число элементов в сообщении;
- **datatype** – тип пересылаемых данных;
- **comm** – коммуникатор, в рамках которого идет обмен;
- **size** – размер буфера в байтах.

Вычисление размера буфера

```
MPI_Pack_size(20, type1, comm, &s1);
MPI_Pack_size(40, type2, comm, &s2);
size = s1 + s2 + 2 * MPI_BSEND_OVERHEAD;
...
buf = allocate size bytes
...
MPI_Buffer_attach(buffer, size);
...
MPI_Bsend(..., count=20, datatype=type1, ...);
...
MPI_Bsend(..., count=40, datatype=type2, ...);
```

Для чего `MPI_Buffer_detach` возвращает размер и указатель на буфер сообщений?

Одновременно в MPI можно работать только с одним буфером. Если какая-то сторонняя библиотека использует буфер, она может отключить текущий буфер, произвести обмен и подключить старый буфер.

Отключение и подключение буфера

```
int size, mysize, idummy;  
void *ptr, *myptr, *dummy;  
...  
MPI_Buffer_detach(&ptr, &size);  
MPI_Buffer_attach(myptr, mysize);  
...  
... library code ...  
...  
MPI_Buffer_detach(&dummy, &idummy);  
MPI_Buffer_attach(ptr, size);
```

Отсылка по готовности

- Завершение гарантирует безопасность буфера отсылаемого сообщения.
- Не гарантирует завершение приема.
- Синтаксис аналогичен MPI_Send.
- Корректность работы гарантируется только если **до** вызова уже была выставлена соответствующая операция приема.
- Иначе — неопределенное поведение (ошибка, зависание, потеря данных — зависит от реализации).
- В ряде случаев может работать быстрее из-за упрощенного протокола.

Базовая функция приема

- Завершение гарантирует, что все данные приняты.
- Может принять данные от любой функции отсылки.
- Может принять меньше данных, чем указано, но не может больше.

```
int MPI_Recv(void *buf, int count, MPI_Datatype datatype, int source,  
int tag, MPI_Comm comm, MPI_Status *status);
```

- **buf** – адрес начала буфера приема;
- **count** – максимальное число элементов в буфере;
- **datatype** – MPI-тип принимаемых данных;
- **source** – номер процесса-отправителя в группе, связанной с коммуникатором **comm**;
- **tag** – идентификатор сообщения;
- **comm** – коммуникатор;
- **status** – статус принятого сообщения.

Специальная структура, хранящая статус принятого сообщения. В ней описываются размер принятого сообщения, номер процесса-отправителя и тег сообщения.

Поля структуры:

- **MPI_SOURCE** – номер процесса-отправителя;
- **MPI_TAG** – тег принятого сообщения;
- **MPI_ERROR** – код ошибки (для некоторых коллективных/множественных wait).

Размер реально принятого сообщения можно узнать через функцию `MPI_Get_count`.

```
int MPI_Get_count(MPI_Status *status, MPI_Datatype datatype, int  
*count)
```

- **status** – статус сообщения, размер которого требуется узнать;
- **datatype** – в каких типах требуется размер;
- **count** – размер сообщения в типах **datatype**. Если размер сообщения не кратен типу **datatype**, то вернется **MPI_UNDEFINED**.

- **MPI_ANY_SOURCE** – может быть указана вместо аргумента **source**, тогда сообщение будет принято от любого процесса в коммутаторе **comm**;
- **MPI_ANY_TAG** – может быть указана в качестве аргумента **tag**, будет принято сообщение с любым тегом;
- **MPI_STATUS_IGNORE** – может быть указано вместо аргумента **status**, тогда статус сообщения будет проигнорирован.

Возможны любые комбинации используемых констант.

Примеры

Прием числа

```
int a;  
...  
MPI_Recv(&a, 1, MPI_INT, rank, tag,  
         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

Прием массива

```
int a[5]; (или динамический)  
...  
MPI_Recv(a, 5, MPI_INT, rank, tag,  
         MPI_COMM_WORLD, MPI_STATUS_IGNORE);  
или  
MPI_Recv(&a[0], 5, MPI_INT, rank, tag,  
         MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

- Типы данных в соответствующих операциях приема и отсылки должны совпадать.
- Исключение составляет `MPI_PACKED` (упакованные данные).
- Для отправки произвольных данных можно использовать `MPI_BYTE`, тогда данные будут пересылаться в неизменном двоичном виде.
- На однородном кластере предпочтительнее типизированные данные (`MPI_INT` и т.п.), а не «сырые» байты.

Корректная работа

```
int *a, *b;  
...  
MPI_Comm_rank(comm, &rank);  
if (rank == 0) {  
    MPI_Send(a, 5, MPI_INT, 1, tag, comm);  
} else {  
    MPI_Recv(b, 10, MPI_INT, 0, tag, comm, &st);  
}
```

Типы **a**, **b** должны быть **int**, а размер буфера приема — не менее **5** элементов.

Некорректная работа

```
int *a, *b;  
...  
MPI_Comm_rank(comm, &rank);  
if (rank == 0) {  
    MPI_Send(a, 5, MPI_INT, 1, tag, comm);  
} else {  
    MPI_Recv(b, sizeof(int) * 10, MPI_BYTE, 0, tag, comm,  
}
```

Типы на стороне send и recv **не совпадают**.

Корректная работа

```
int *a, *b;  
...  
MPI_Comm_rank(comm, &rank);  
if (rank == 0) {  
    MPI_Send(a, sizeof(int) * 5, MPI_BYTE, 1, tag, comm);  
} else {  
    MPI_Recv(b, sizeof(int) * 10, MPI_BYTE, 0, tag, comm,  
}
```

Пример пересылок

```
int rank, count;
char buf[100];
MPI_Status status;
MPI_Comm_rank(comm, &rank);
if (rank == 0) {
    strcpy(buf, "Hello from 0");
    MPI_Send(buf, strlen(buf) + 1, MPI_CHAR, 1, 99, comm);
} else if (rank == 1) {
    MPI_Recv(buf, 100, MPI_CHAR, MPI_ANY_SOURCE,
             MPI_ANY_TAG, comm, &status);
    MPI_Get_count(&status, MPI_CHAR, &count);
    printf("Message '%s', from %d, tag %d, size %d\n",
           buf,
           status.MPI_SOURCE,
           status.MPI_TAG,
           count);
}
```

Взаимная блокировка (deadlock)

Оба процесса ждут друг друга — типичная ошибка в р2р.

Опасно (оба блокируются на Send при крупных сообщениях / Ssend):

- процесс 0: Send → Recv
- процесс 1: Send → Recv

Безопаснее:

- разные порядки: один Send→Recv, другой Recv→Send;
- неблокирующие Isend/Irecv + Wait/Waitall;
- **MPI_Sendrecv** — атомарный обмен «отправить и принять».

Deadlock: антипример

```
/* Оба процесса: */  
MPI_Send(sbuf, n, MPI_INT, neighbor, tag, comm);  
MPI_Recv(rbuf, n, MPI_INT, neighbor, tag, comm, &st);  
/* При rendezvous / Ssend -- взаимная блокировка. */
```

На малых сообщениях (eager) код «случайно» работает — поэтому баг часто всплывает только на больших размерах или другой реализации MPI.

MPI_Sendrecv

```
int MPI_Sendrecv(const void *sendbuf, int sendcount, MPI_Datatype  
sendtype, int dest, int sendtag, void *recvbuf, int recvcount,  
MPI_Datatype recvtype, int source, int recvtag, MPI_Comm comm,  
MPI_Status *status);
```

Обмен с соседом (кольцо)

```
int left = (rank - 1 + size) % size;  
int right = (rank + 1) % size;  
MPI_Sendrecv(sbuf, n, MPI_INT, right, tag,  
             rbuf, n, MPI_INT, left, tag,  
             comm, MPI_STATUS_IGNORE);
```

Есть также `MPI_Sendrecv_replace` — обмен «на месте» в одном буфере.

Неблокирующие взаимодействия

- Существуют неблокирующие аналоги всех базовых функций взаимодействия.
- Аргументы у функций аналогичные, за исключением последнего аргумента — идентификатора неблокирующего взаимодействия **request**.
- Вызов только *инициирует* операцию; завершение — через Wait/Test.

Тип данных для идентификатора: **MPI_Request**.

```
int MPI_Isend(void *buf, int count, MPI_Datatype datatype, int dest, int  
tag, MPI_Comm comm, MPI_Request *request);
```

```
int MPI_Irecv(void *buf, int count, MPI_Datatype datatype, int source,  
int tag, MPI_Comm comm, MPI_Request *request);
```

Аналогично для других типов взаимодействия.

Пока неблокирующая операция не завершена, буфер **трогать нельзя**:

- после `Isend` — не изменять `send`-буфер до `Wait/Test`;
- после `Irecv` — не читать `recv`-буфер до `Wait/Test`.

Антипример

```
MPI_Isend(buf, n, MPI_INT, dest, tag, comm, &req);  
buf[0] = 42;           /* ОШИБКА: буфер еще «в полете» */  
MPI_Wait(&req, MPI_STATUS_IGNORE);
```

Проверка завершения

Завершение неблокирующего взаимодействия аналогично завершению его блокирующего аналога. Для проверки завершения операции существуют процедуры `MPI_Test` и `MPI_Wait`.

Процедура `MPI_Wait` дожидается завершения неблокирующего взаимодействия. Последовательный вызов неблокирующей операции и `MPI_Wait` аналогичен соответствующему блокирующему вызову.

Процедура `MPI_Test` проверяет статус завершения (завершено или нет) без блокировки.

`int MPI_Wait(MPI_Request *request, MPI_Status *status)`

- **request** – хендлер неблокирующего взаимодействия;
- **status** – статус; полезен в первую очередь для операций **приема**.
Можно использовать **MPI_STATUS_IGNORE**.

`int MPI_Test(MPI_Request *request, int *flag, MPI_Status *status)`

- **request** – хендлер неблокирующего взаимодействия;
- **flag** – флаг завершения, 1 – завершено, 0 – не завершено;
- **status** – статус (для приема); можно использовать **MPI_STATUS_IGNORE**.

Проверка завершения нескольких операций

Существуют аналоги `MPI_Test` и `MPI_Wait` для нескольких операций.

Проверка	Wait	Test
Как минимум одна	<code>MPI_Waitany</code>	<code>MPI_Testany</code>
Все	<code>MPI_Waitall</code>	<code>MPI_Testall</code>
Какие-либо (несколько)	<code>MPI_Waitsome</code>	<code>MPI_Testsome</code>

На практике чаще всего достаточно `MPI_Wait` / `MPI_Waitall`.

```
int MPI_Waitall(int count, MPI_Request array_of_requests[],  
MPI_Status array_of_statuses[]);
```

- **count** – размер массива **array_of_requests**;
- **array_of_requests** – массив с хендлерами неблокирующих взаимодействий;
- **array_of_statuses** – массив статусов; можно использовать **MPI_STATUSES_IGNORE**.

Дождается выполнения всех взаимодействий. Типичный паттерн: несколько `Isend/Irecv`, затем один `Waitall`.

Остальные Wait/Test (кратко)

- `MPI_Waitany` / `MPI_Testany` — ждать/проверить *одну* любую завершённую операцию (возвращают `index`).
- `MPI_Waitsome` / `MPI_Testsome` — ждать/проверить *несколько* завершённых; `outcount` — сколько завершилось (`Testsome`: может быть 0).
- Сигнатуры — в документации MPI; логика та же, что у `Wait/Test`, но для массива `request`.

Иногда размер сообщения заранее неизвестен.

- MPI_Probe — блокирующе ждет подходящее сообщение, **не принимая** его.
- MPI_Iprobe — неблокирующая проверка наличия сообщения.
- По status через MPI_Get_count узнаем размер, выделяем буфер, затем MPI_Recv.

Пример: Probe

```
MPI_Status st;
int count;
MPI_Probe(MPI_ANY_SOURCE, tag, comm, &st);
MPI_Get_count(&st, MPI_INT, &count);
int *buf = malloc(count * sizeof(int));
MPI_Recv(buf, count, MPI_INT,
         st.MPI_SOURCE, st.MPI_TAG, comm, &st);
```

В MPI-3 есть также matched probe: MPI_Mprobe / MPI_Improbe + MPI_Mrecv (сообщение «бронируется» и не будет перехвачено другим Recv).

Если одна и та же пересылка повторяется много раз (цикл по времени):

- один раз создать request: `MPI_Send_init`, `MPI_Recv_init`, ...
- многократно: `MPI_Start` / `MPI_Startall`, затем `Wait`/`Waitall`;
- в конце: `MPI_Request_free`.

Экономия на повторной инициализации операции; семантика завершения — как у неблокирующих аналогов.

- **Large count (MPI-3/4):** тип `MPI_Count` и варианты `*_c` — когда `int count` не хватает.
- **Matched probe (MPI-3):** `Mprobe/Mrecv` — безопасный прием при гонках нескольких `Recv`.
- **Partitioned p2p (MPI-4):** постепенная отправка частей сообщения (для гибридных `MPI+threads`) — за рамками семинара.

Задание 1

Задание к следующему семинару. Прислать на почту mipt.courses@gmail.com в формате pdf и исходный код.

Измерение латентности передачи сообщений (1 балл)

Построить график времени передачи сообщения от его размера (подобрать масштаб для лучшего отображения). Размеры сообщения от 1 Б до 1 МБ. Построить для типов сообщений MPI_Send, MPI_Ssend, MPI_Bsend.

Измерение скорости передачи сообщений (1 балл)

Построить график скорости (ед. данных в ед. времени) передачи сообщения от его размера. Размеры сообщения от 1 Б до 1 МБ. Построить для типов сообщений MPI_Send, MPI_Ssend, MPI_Bsend.

Методика измерения (рекомендуется)

- Схема **ping-pong**: $0 \leftrightarrow 1$; время одного направления $\approx$ half round-trip.
- Прогрев (*warmup*): несколько «холостых» обменов до замеров.
- Много повторов; на график — среднее или медиана (указать, что именно).
- Размеры по логарифмической шкале (1, 2, 4, ... Б или близко к степеням двойки).
- Для MPI_Bsend: заранее Buffer_attach достаточного размера.
- Указать реализацию MPI и число процессов/узлов.

Спасибо за внимание! Вопросы?